CYBERSHIELD AI: A WEB-BASED REAL-TIME CYBERSECURITY MONITORING DASHBOARD WITH AI-ASSISTED THREAT ANALYSIS

Project Phase I Report

submitted to the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the degree

Bachelor of Technology

in

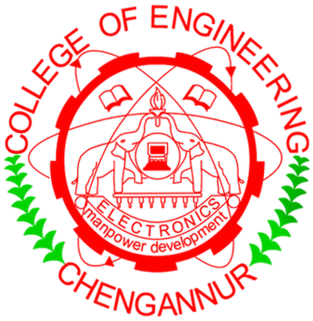
Computer Science and Engineering

by

SREEDEVI R P (CHN23CS181)

VISHNU V NAIR (CHN23CS196)

ABHI RAJ (LCHN23CS197)



Department of Computer Engineering

College of Engineering Chengannur Kerala - 689121

Phone: (0479) 24541125 Fax: (0479) 2451424

Website: www.ceconline.edu

OCTOBER 2026

DEPARTMENT OF COMPUTER ENGINEERING

COLLEGE OF ENGINEERING CHENGANNUR

2026-27

CERTIFICATE

This is to certify that the project report titled ***CYBERSHIELD AI: A WEB-BASED REAL-TIME CYBERSECURITY MONITORING DASHBOARD WITH AI-ASSISTED THREAT ANALYSIS*** *is a bonafide record of the* ***CSD 415 PROJECT PHASE I*** *presented by* ***SREEDEVI R P (CHN23CS181), VISHNU V NAIR (CHN23CS196) & ABHI RAJ (LCHN23CS197)****, Seventh Semester B.Tech. students in Computer Science and Engineering, under our guidance and supervision. This report is submitted in partial fulfillment of the requirements for the award of the degree* ***B.Tech. in Computer Science and Engineering*** *of* ***APJ Abdul Kalam Technological University****.*

Smt.Varsha C Mohan
(Project Guide)
Assistant Professor
Dept. of Computer Engineering
College of Engineering Chengannur

Dr. Geetha S
(Project Coordinator)
Professor
Dept. of Computer Engineering
College of Engineering Chengannur

Dr. Renu George
Head of the Department
Dept. of Computer Engineering
College of Engineering Chengannur

DECLARATION

We undersigned hereby declare that the project report **"CyberShield AI: A Web-Based Real-Time Cybersecurity Monitoring Dashboard with AI-Assisted Threat Analysis"**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under the supervision of **Varsha C Mohan**, Assistant Professor, Department of Computer Engineering. This submission represents our ideas in our own words, and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma, or similar title of any other University.

Sreedevi R P (CHN23CS181)

Place: Chengannur

Vishnu V Nair (CHN23CS196)

Date: 14/10/2026

Abhi Raj (LCHN23CS197)

ACKNOWLEDGEMENT

This work would not have been possible without the support and guidance of many people. First and foremost, we thank Almighty God for providing us with the strength, resources, and ability to successfully carry out this project.

We would like to express our sincere gratitude to **Prof. Dr. Hari V S**, The Principal of College of Engineering Chengannur, for providing excellent facilities and creating a conducive environment for the successful completion and presentation of this project. Special thanks also go to HOD **Dr. Renu George**, Department of Computer Engineering, for the exceptional support, guidance, and encouragement throughout the project.

We are deeply thankful to our project guide **Smt.Varsha C Mohan**, Assistant Professor, Department of Computer Engineering, for her valuable advice, constant guidance, and support throughout the project, and to our project coordinator **Dr. Geetha S**, Department of Computer Engineering, for her timely suggestions.

Finally, we would like to thank our friends and faculty members for their cooperation, encouragement, and moral support, without which this project would not have been completed successfully. Thank you all for your understanding and support.

Sreedevi R P

CHN23CS181

Vishnu V Nair

CHN23CS196

Abhi Raj

LCHN23CS197

ABSTRACT

Phishing and malware remain the most common ways in which attackers gain access to systems, yet the tools used to detect them are usually scattered across separate dashboards, return unexplained verdicts, and are costly for small organisations to operate. This project presents **CyberShield AI**, a lightweight web-based Security Operations Center (SOC) platform that unifies phishing detection, malware forensics, network telemetry, threat intelligence and alerting in a single interface. Phishing URLs are analysed by a dual-engine detector: the PhishGuard model, developed by the project team and trained on a corpus of 235,000 verified URLs, is accessed through its API, and a locally trained Random Forest classifier works on 15 lexical and structural features including Shannon entropy. A consensus engine compares the two verdicts, reports full agreement or discordance, and computes a combined risk score. Suspicious files are hashed in the browser with SHA-256 using the Web Crypto API and the hash is checked against 72 or more antivirus engines through VirusTotal, so the file itself is not uploaded. The Google Gemini API converts technical indicators into short plain-language explanations. Threat intelligence from public feeds, simulated network telemetry and automated Discord and Slack webhook alerts complete the platform. Users may work as guests or sign in with Google or email and password; signed-in users receive alerts and administrators manage roles and webhooks. The system is implemented with React, TypeScript and Vite on the front end, a Node.js and Express back end, Python and scikit-learn for model training, and Firebase Firestore for persistence. This Phase I report presents the motivation, literature survey, requirements, design, methodology and results obtained so far.

CONTENTS

Declaration ii

Acknowledgement iii

Abstract iv

List of Figures viii

List of Tables ix

Abbreviations x

1 INTRODUCTION 1

1.1 Project Area 2

1.2 Objectives 2

2 PROBLEM DEFINITION AND MOTIVATION 3

2.1 Existing System 3

2.1.1 Limitations of the Existing System 3

2.2 Problem Statement 3

2.3 Need for the Proposed System 4

2.4 Proposed Solution 4

3 LITERATURE REVIEW 5

3.1 XGBoost-Based URL Phishing Detection with Cross-Dataset Validation (2026) 5

3.2 ChatSpamDetector: Large Language Model-Based Phishing Email Detection (2026) 5

3.3 Dynamic Phishing URL Detection with Crowdsourced Threat Intelligence (2026) 6

3.4 Early Detection of Phishing URLs in Parked Domains (2025) 6

3.5 Hybrid Feature Image Mapping for Malware Detection (2026) 6

3.6 Application of Deep Learning Models for Real-Time Malware Detection (2024) 6

3.7 Summary of the Reviewed Studies 7

4 REQUIREMENT ANALYSIS **9**

4.1 Hardware Requirements 9

4.2 Software Requirements 9

4.3 Functional Requirements 10

4.4 Non-Functional Requirements 11

5 DESIGN AND IMPLEMENTATION **13**

5.1 Design 13

5.1.1 Use Case Diagram 13

5.1.2 Data Flow Diagram 14

5.1.2.1 Level-0 Data Flow Diagram 14

5.1.2.2 Level-1 Data Flow Diagram 15

5.1.3 System Architecture 16

5.1.3.1 Client Side 16

5.1.3.2 Server Side 16

5.1.3.3 Models, External Services and Storage 16

5.1.4 Methodology 17

5.1.4.1 Data Preparation 17

5.1.4.2 URL Feature Engineering 17

5.1.4.3 Random Forest Classifier 18

5.1.4.4 Dual-Engine Consensus 19

5.1.4.5 Malware Forensics 19

5.1.4.6 Explainable AI 19

5.1.4.7 Threat Intelligence and Network Telemetry 19

5.1.4.8 Alerting 19

5.1.4.9 Authentication and Access Control 19

5.2 Implementation 20

5.2.1 Web Application Development 20

5.2.2 Testing and Evaluation Plan 21

6 RESULTS AND DISCUSSION **22**

6.1 Evaluation Metrics 22

6.2 Experimental Set-up 23

6.3 Detection Performance 24

6.4 System Latency and Alert Delivery 24
6.5 Screenshots of the Application 25
6.6 Testing and Validation of the Application 27
6.7 Discussion 27

7 CONCLUSION AND FUTURE SCOPE **29**
7.1 Conclusion 29
7.2 Future Scope 29

REFERENCES **31**

LIST OF FIGURES

5.1 Use case diagram 14
5.2 Data flow diagram – Level 0 15
5.3 Data flow diagram – Level 1 15
5.4 System architecture of CyberShield AI 17

6.1 Application screens: sign-in, SOC dashboard, phishing detection and malware forensics 25
6.2 Application screens: network telemetry, threat intelligence, security alerts and settings 26

LIST OF TABLES

3.1 Summary of the reviewed literature 7

5.1 The 15 URL features 18

5.2 Pages of the web application 20

6.1 Experimental set-up 23

6.2 Detection performance of the phishing engines 24

6.3 System latency and alert delivery 24

6.4 Functional and system test cases 27

ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
AV	Antivirus
CVE	Common Vulnerabilities and Exposures
DFD	Data Flow Diagram
DGA	Domain Generation Algorithm
HTTPS	HyperText Transfer Protocol Secure
IOC	Indicator of Compromise
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
OTP	One-Time Password
PE	Portable Executable
RBAC	Role-Based Access Control
REST	Representational State Transfer
RF	Random Forest
SHA	Secure Hash Algorithm
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation and Response
SOC	Security Operations Center
UI	User Interface
URL	Uniform Resource Locator
XAI	Explainable Artificial Intelligence

CHAPTER 1

INTRODUCTION

Phishing and malware are among the most common ways in which attackers gain an initial foothold in a network. Phishing relies on deceptive links, forged web pages and messages that persuade users to reveal credentials or run harmful files, while modern malware uses packing, obfuscation and fileless techniques to avoid signature-based detection [1, 4, 5]. A growing body of research applies machine learning to these problems, covering URL classification, parked and generated domains, malicious-email analysis and malware detection from file features [1, 2, 3, 4, 5, 6].

Detection alone, however, does not make an organisation secure. Security analysts must work with many separate tools, one for phishing, another for malware, another for network traffic and yet another for threat feeds, and must watch each of them for alerts. Large organisations address this with Security Information and Event Management (SIEM) and Security Orchestration, Automation and Response (SOAR) platforms, but these require significant computing resources, complex set-up and expensive licences. Small and medium enterprises and junior analysts are therefore left with fragmented tools or no tooling at all.

Machine learning models in published studies often report high accuracy on offline benchmarks, but they are seldom delivered as part of a working operational workflow. Most return only a label or a numerical score, giving the analyst no reason why an artefact was flagged, and many do not connect to any mechanism for notifying the team when a threat is found [1, 2, 6]. Explainable AI, in which the reasoning behind a verdict is stated in plain language, is increasingly seen as necessary for analysts to trust and act on automated decisions [2].

This project, **CyberShield AI**, develops a lightweight web-based SOC platform that unifies real-time telemetry, automated detection, threat intelligence aggregation and alert dispatch in a single command centre. Phishing URLs are checked by two independent models whose verdicts are cross-verified; files are examined through browser-side hashing and multi-engine lookup; the Google Gemini API explains flagged threats in plain English; and configurable webhooks push alerts to Discord and Slack. The platform can be used as a guest or after signing in, and administrators control roles and integrations.

1.1 Project Area

The project lies in the fields of cybersecurity, applied machine learning and explainable artificial intelligence, combined with full-stack web application development. It applies a Random Forest classifier on engineered URL features, a cloud-hosted phishing model developed by the project team, client-side cryptographic hashing, large-language-model explanations and webhook-based alerting. The front end is built with React, TypeScript and Vite, the back end with Node.js and Express, the machine learning components with Python and scikit-learn, and the data is stored in Firebase Firestore.

1.2 Objectives

The main objectives of CyberShield AI are:

1. **Dual-engine phishing detection:** To classify URLs with two independent engines, the PhishGuard model and a local Random Forest classifier, and to compare their verdicts through a consensus engine.

2. **Explainable verdicts:** To use the Google Gemini API to explain each flagged threat in short, plain-language summaries that an analyst can act on.

3. **Lightweight malware forensics:** To compute SHA-256 hashes in the browser and correlate them with 72 or more antivirus engines without uploading the file.

4. **Unified monitoring:** To bring phishing, malware, network telemetry, threat intelligence and alerts into one dashboard.

5. **Threat intelligence aggregation:** To collect vulnerabilities, malicious indicators and phishing data from public feeds and make them searchable.

6. **Automated alerting:** To dispatch alerts of configurable severity to Discord and Slack through webhooks.

7. **Flexible access:** To allow use as a guest and, after sign-in, to provide alerts, role-based access control and administrative functions.

8. **Low cost and easy deployment:** To keep the platform lightweight so that small teams can run it without enterprise infrastructure.

CHAPTER 2

PROBLEM DEFINITION AND MOTIVATION

2.1 Existing System

At present, the detection of phishing and malware in most small organisations depends on a collection of separate tools: browser warnings or reputation services for URLs, antivirus or online scanners for files, network monitors for traffic and public websites for threat feeds. Analysts move between these tools manually and combine the results themselves. Enterprise SIEM and SOAR platforms unify these functions, but they are designed for large security teams and budgets.

2.1.1 Limitations of the Existing System

- Detection tools are fragmented, so analysts must switch between separate dashboards for phishing, malware, network traffic and threat feeds.
- Many machine learning detectors return only a label or a score and do not explain why a URL or file was flagged.
- SIEM and SOAR platforms need substantial computing resources, complex configuration and costly licences.
- Alerts are often not forwarded automatically to team channels, so threats may remain unnoticed outside working hours.
- Uploading suspicious files to online scanners can expose confidential content.
- Models that perform well on offline benchmarks may adapt slowly to new threats, and published systems are often not packaged for operational use [1, 3, 6].

2.2 Problem Statement

Existing phishing and malware detection tools give unexplained verdicts, sit in separate dashboards and alert users too late, so small organisations and junior analysts have no affordable, unified, real-time platform that detects threats, explains them and notifies users promptly.

2.3 Need for the Proposed System

A single lightweight platform that combines detection, explanation, threat intelligence and alerting can reduce the effort needed to triage threats and can make a basic security operations capability available to teams that cannot afford enterprise suites. Cross-verifying two detection engines reduces the chance that a single model's mistake goes unnoticed, and explanations written in plain language help junior analysts decide what to do. Processing file hashes in the browser protects confidential files, and automated webhooks ensure that the right people are informed without having to watch a dashboard continuously.

2.4 Proposed Solution

CyberShield AI is a web-based platform with the following main components.

1. **Two-way phishing detection engine:** Each URL is analysed by Engine 1, the PhishGuard model (developed by the project team and trained on 235,000 verified URLs, accessed through an API), and Engine 2, a Random Forest classifier operating on 15 lexical and structural features. A consensus engine reports full agreement or a discordant verdict and computes a combined risk score.

2. **AI SOC co-pilot:** The Google Gemini API converts raw indicators into short, actionable summaries.

3. **In-browser malware forensics:** A file's SHA-256 hash is computed in the browser and checked against 72 or more antivirus engines through VirusTotal; the file is not uploaded.

4. **Network telemetry:** Simulated network traffic is displayed on live charts with protocol distribution and anomaly indicators.

5. **Threat intelligence:** Vulnerabilities, indicators and phishing data from public feeds are aggregated and can be filtered instantly.

6. **Security alerts:** Alerts of configurable severity are queued in the dashboard and sent to Discord and Slack through webhooks.

7. **Access control:** Users may use the platform as guests or sign in with Google or email and password. Signed-in users receive alerts, and administrators manage roles and integration settings.

CHAPTER 3

LITERATURE REVIEW

This chapter reviews six selected studies on automated detection of phishing and malware. They cover URL classification, domain-based early detection, large-language-model analysis, crowdsourced threat intelligence, image-based malware classification and real-time deep learning. For each study, the methodology, the limitation relevant to this project and the way CyberShield AI addresses it are described. Numerical results quoted are those reported by the respective authors on their own data and are not directly comparable with each other.

3.1 XGBoost-Based URL Phishing Detection with Cross-Dataset Validation (2026)

Misiek and Hyla proposed a phishing detector for URLs based on the XGBoost gradient-boosting algorithm [1]. Lexical features are extracted from each URL and used to train a boosted-tree classifier. The distinguishing feature of the study is cross-dataset validation: the model is trained on one dataset and tested on others to check that it generalises beyond the data it was built on. The work shows that gradient-boosted trees on URL features can be accurate, but it presents a classifier rather than a system and has no mechanism to notify the security team of a detection. CyberShield AI attaches multi-channel webhook alerts to its detectors.

3.2 ChatSpamDetector: Large Language Model-Based Phishing Email Detection (2026)

Koide et al. use a large language model to judge phishing emails [2]. The email is parsed and decoded, simplified, and turned into a prompt that guides the model through a chain of reasoning; the model returns a verdict and its reasons in a structured form. In the authors' evaluation, GPT-4 gave the best results, with an accuracy of 99.70%. The study shows that an LLM can supply the reasoning behind a verdict, but it targets emails only and is not part of a monitoring workflow that also covers URLs, files, telemetry and alerts. CyberShield AI uses the Gemini API in a similar spirit to turn its own indicators into plain-language explanations inside a unified dashboard.

3.3 Dynamic Phishing URL Detection with Crowdsourced Threat Intelligence (2026)

Aldhuwayhi and Alromih propose a phishing URL detection system that combines machine learning with crowdsourced threat intelligence [3]. Verdicts from the model are supplemented by community-sourced reports, so the system can adapt to new phishing campaigns. The approach addresses concept drift, the tendency of a static model to lose accuracy against previously unseen attacks. CyberShield AI adopts the same principle through a human-in-the-loop verification step in which analysts can confirm or discard a flagged threat, together with live threat-intelligence feeds.

3.4 Early Detection of Phishing URLs in Parked Domains (2025)

Duarte et al. study phishing URLs hosted on parked domains and apply machine learning to detect them early [4]. Features are extracted from the domains and their URLs and used to train a classifier that flags suspicious domains before they are weaponised. Domains produced by domain generation algorithms and obfuscated names are a known weakness of static blacklists. CyberShield AI includes character-level Shannon entropy and structural URL features within its 15-feature set to detect random-looking and obfuscated domain names.

3.5 Hybrid Feature Image Mapping for Malware Detection (2026)

Elgarduh et al. proposed Hybrid Feature Image Mapping (HFIM), which converts several kinds of information about a program into one multi-channel image [5]. Structural information from the binary content, semantic information from opcode transitions and behavioural information from API activity are mapped into the channels of an RGB image, which is classified by convolutional neural networks; seven network architectures were compared. The approach is accurate but computationally heavy, which makes it impractical to run inside a browser. CyberShield AI instead computes a SHA-256 hash locally in the browser and uses multi-engine lookup, which needs very little computation on the client.

3.6 Application of Deep Learning Models for Real-Time Malware Detection (2024)

Gutierrez et al. applied deep learning to automatic malware detection with a focus on real-time operation [6]. Deep models are trained on features extracted from malware samples and evaluated for accuracy as well as inference speed. Models of this kind can introduce latency

that makes interactive use difficult. CyberShield AI uses a dual-engine design in which an instant local lexical score is shown at once while cloud analysis and the AI explanation are completed asynchronously.

3.7 Summary of the Reviewed Studies

Study	**Methodology**	**Gap identified**	**CyberShield AI resolution**
Misiek & Hyla [1]	XGBoost on URL features; cross-dataset validation	No live alert dispatcher	Discord/Slack webhooks
Koide et al. [2]	LLM with chain-of-thought prompt on parsed email	Emails only; not part of a monitoring workflow	Gemini explanations inside a unified dashboard
Aldhuwayhi & Alromih [3]	ML on URL features plus crowdsourced threat intelligence	Concept drift on zero-day threats	Human verification loop and live feeds
Duarte et al. [4]	Features from parked domains with ML classifier	DGA domains evade blacklists	15 features with Shannon entropy
Elgarduh et al. [5]	Multi-channel image of structural, semantic and behavioural features; CNNs	Heavy computation, unsuitable for browser	Browser SHA-256 and 72+ engine lookup
Gutierrez et al. [6]	Deep learning on malware features with real-time evaluation	Inference latency affects the interface	Instant lexical score plus asynchronous cloud AI

Table 3.1: Summary of the reviewed literature

The six studies concentrate mainly on the accuracy of individual detection models. Aspects of an operational security workflow, such as explaining each verdict in plain language, combining phishing, malware, telemetry and threat intelligence in one place, and alerting the team automatically, receive less attention. CyberShield AI combines a dual-engine phishing detector,

browser-side malware hashing, Gemini explanations and webhook alerting in one lightweight web platform.

CHAPTER 4

REQUIREMENT ANALYSIS

This chapter presents the hardware, software, functional and non-functional requirements of the CyberShield AI system. The requirements are derived from the objectives and proposed workflow of the project.

4.1 Hardware Requirements

The following hardware requirements are considered for the development, deployment and use of CyberShield AI.

- **Development and training machine:**
  - **Processor:** Intel Core i5/i7 (8th generation or later), AMD Ryzen 5 or Apple Silicon.
  - **RAM:** At least 8 GB; 16 GB is recommended for model training and concurrent development servers.
  - **Storage:** A 256 GB SSD with at least 5 GB free for Node modules, the Python environment and datasets.
  - **Network:** An internet connection of at least 10 Mbps for calls to external APIs.
- **Production deployment:** A cloud virtual machine or container with 1 vCPU and 1 to 2 GB of RAM (for example Google Cloud Run, Render or AWS EC2 t3.small), and 10 GB of persistent storage or a managed database such as Firestore.
- **Client device:** A desktop, laptop or tablet with a dual-core processor, 4 GB of RAM and a modern web browser.

4.2 Software Requirements

The software requirements of the CyberShield AI system include the following components.

- **Operating system:** Ubuntu Linux 22.04 LTS, macOS or Windows 10/11.

- **Frontend:** React 18 with TypeScript, Vite 6, Tailwind CSS v4, Recharts for charts and Lucide React for icons.
- **Backend:** Node.js (version 18 or 20 LTS) with TypeScript, Express.js 4 and Axios for HTTP requests.
- **Machine learning:** Python 3.10 or later with scikit-learn, Pandas, NumPy, Joblib and tldextract for training the Random Forest model and extracting features.
- **AI services:** Google Gemini generative AI SDK, the PhishGuard model API and the VirusTotal API.
- **Database and cloud:** Firebase Firestore, Google Cloud Run or Render for hosting, and Discord and Slack webhook APIs for alerts.
- **Web browser support:** Current versions of Google Chrome, Mozilla Firefox, Microsoft Edge and Safari.

4.3 Functional Requirements

The functional requirements describe the operations and services provided by the CyberShield AI system.

FR-01 Centralised Security Dashboard

The system shall present system status, the number of scans, phishing threats found, alert severity and live telemetry in a single dashboard.

FR-02 Phishing URL Analysis

The system shall accept a URL, analyse it with both detection engines, and display the verdict of each engine, the consensus result, a risk score from 0 to 100, the severity and an AI-generated explanation.

FR-03 Malware Forensics

The system shall compute the SHA-256 hash of a selected file in the browser, evaluate the byte entropy, and display the result of matching the hash against multiple antivirus engines.

FR-04 Network Telemetry

The system shall display live network events, bandwidth, protocol distribution and abnormal traffic indicators.

FR-05 Threat Intelligence

The system shall collect and display vulnerabilities, malicious IP addresses and other indicators from public feeds and shall allow instant filtering.

FR-06 Incident Management and Alerting

The system shall queue security alerts, classify them as low, medium, high or critical, avoid duplicates, and send alerts at or above a configurable threshold to Discord and Slack webhooks.

FR-07 Access Control

The system shall allow guest use and sign-in through Google or email and password with OTP verification, shall send alerts only to signed-in users, and shall restrict administrative actions to the SOC administrator role.

FR-08 Human Verification

The system shall allow an analyst or administrator to verify or discard a flagged threat.

4.4 Non-Functional Requirements

The non-functional requirements specify the quality attributes expected from the CyberShield AI system.

NFR-01 Performance

Client-side hash generation shall complete in under 50 ms for typical files, and dual-engine classification together with the AI explanation shall be delivered in under 2.5 seconds.

NFR-02 Scalability

The system shall be modular so that new detection models or feed parsers can be added independently.

NFR-03 Security and Privacy

File contents shall be hashed locally and shall not leave the browser. Role-based access control shall protect administrative actions, and HTTPS shall be used when the application is deployed remotely.

NFR-04 Usability

The interface shall use a dark theme suited to long monitoring sessions, with a light theme as an alternative, and a colour-coded severity scheme in which green indicates safe, yellow

suspicious and red phishing or critical.

NFR-05 Reliability

The system shall degrade gracefully when an external API is unavailable or slow, using in-memory caching and persistent storage in Firestore, and shall not terminate unexpectedly because of invalid input.

NFR-06 Maintainability

The system shall keep detection, explanation, threat intelligence, alerting and access control as separate modules to ease future changes.

NFR-07 Compatibility

The web application shall work with common modern browsers and with the supported versions of Node.js, Python and the libraries listed above.

NFR-08 Ethical Use

The system shall present its output as decision support for analysts and shall not present it as a final judgement; high-impact actions remain with a human administrator.

CHAPTER 5

DESIGN AND IMPLEMENTATION

CyberShield AI follows a three-tier web architecture made up of a React-based presentation layer, an Express-based application layer and a data and intelligence layer containing the machine learning models, external services and persistent storage. This chapter presents the design of the system, namely the use case diagram, data flow diagrams, system architecture and methodology, followed by its implementation.

5.1 Design

5.1.1 Use Case Diagram

The use case diagram shows the interactions between the actors and the system. The **User** is a security analyst who submits URLs and files, searches threat intelligence, reviews telemetry and takes part in the training simulation. The **SOC Administrator** verifies or discards threats, dispatches alerts and configures the system and role-based access control.

- **Authenticate:** sign in with Google or email and password (optional for guests).
- **Scan URL (phishing):** submit a URL for dual-engine analysis.
- **Scan file (malware):** hash a file in the browser and check the hash against antivirus engines.
- **Search threat intel:** search the aggregated vulnerability and indicator feeds.
- **Generate AI rationale:** obtain a plain-language explanation from the Gemini API.
- **Review live telemetry:** view network traffic and anomaly charts.
- **Security training simulation:** practise recognising threats in simulated scenarios.
- **Verify / discard threat:** confirm or dismiss a flagged threat.
- **Dispatch alert:** send an alert to the configured webhooks.
- **Configure system / RBAC:** manage roles, webhook addresses and alert thresholds.

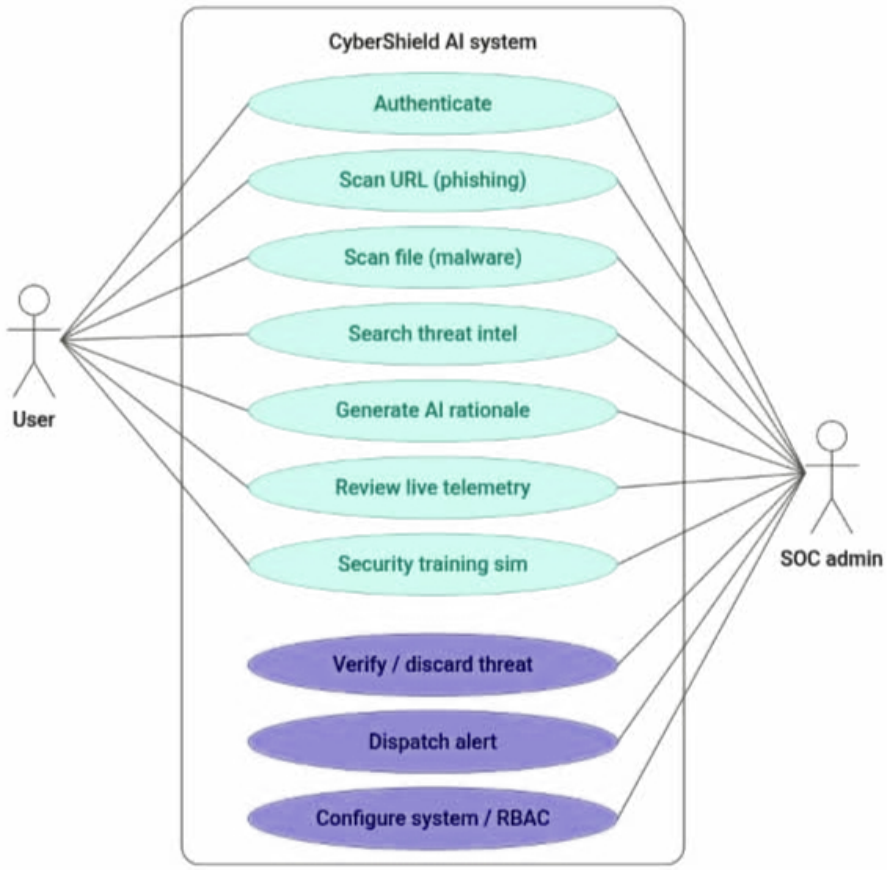


Figure 5.1: Use case diagram

Alternative flow: if a service such as the PhishGuard API or Gemini is unreachable, the system shows the result of the remaining engine and indicates that the explanation is unavailable instead of stopping the analysis.

5.1.2 Data Flow Diagram

5.1.2.1 Level-0 Data Flow Diagram

The context diagram shows CyberShield AI as a single process exchanging data with seven external entities. The User supplies URLs, file hashes and optional sign-in details and receives risk scores, reports, AI explanations and the dashboard, with alerts for signed-in users only. The SOC Administrator supplies role and configuration data and webhook settings and receives alerts, audit logs and system status. Google Sign-In and Firebase Authentication verify identities. The PhishGuard model API returns a phishing probability for URL features, VirusTotal returns detection reports for file hashes, the Gemini API returns explanations, threat intelligence feeds supply vulnerabilities and indicators, and Discord and Slack receive alerts.

5.1.2.2 Level-1 Data Flow Diagram

The Level-1 diagram expands the system into five processes: user and access management (1.0), threat data collection (2.0), threat analysis and detection (3.0), AI explanation and risk scoring (4.0), and alerting and dashboard (5.0). Process 3.0 combines the PhishGuard API and the Random Forest model for URLs with the VirusTotal lookup for file hashes. The data store D1 is Firebase Firestore, which holds user and role data, collected threat data, threat results, risk scores, alerts and audit logs.

Figures 5.2 and 5.3 are shown on the following pages.

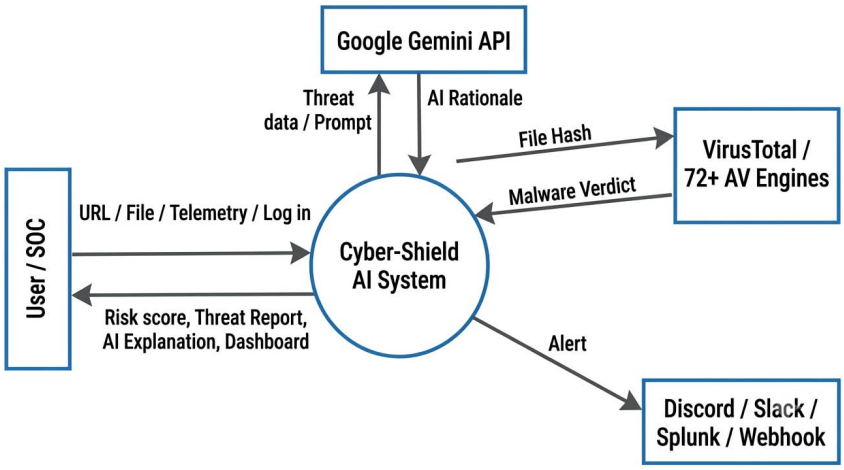


Figure 5.2: Data flow diagram – Level 0

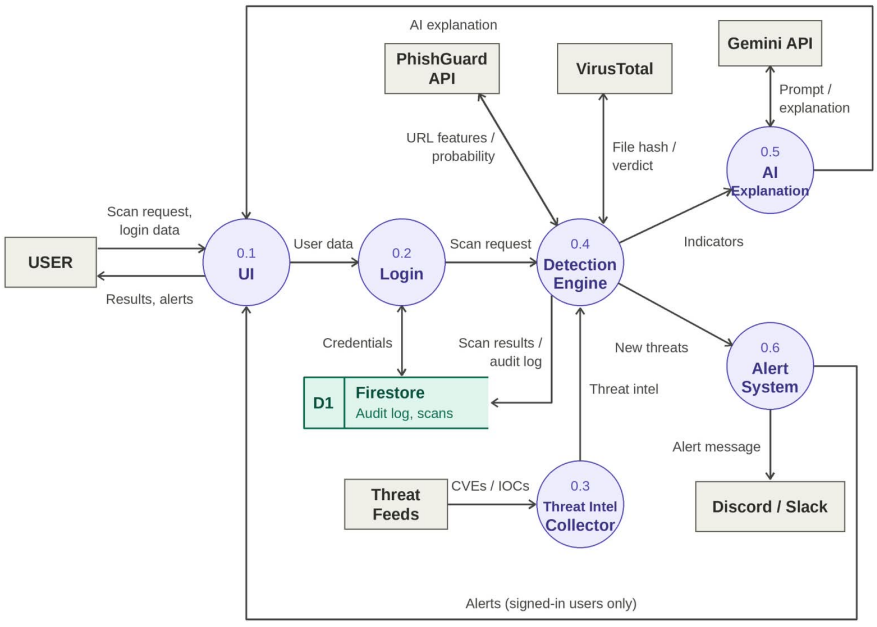


Figure 5.3: Data flow diagram – Level 1

5.1.3 System Architecture

The architecture is organised in layers, shown in Figure 5.4.

5.1.3.1 Client Side

The client is a single-page web application written with React, TypeScript, Vite and Tailwind CSS. A side menu gives access to the SOC dashboard, phishing detection, malware forensics, network telemetry, threat intelligence and security alerts. Recharts is used for telemetry and severity charts.

5.1.3.2 Server Side

A Node.js and Express server written in TypeScript exposes REST endpoints for phishing analysis, malware scanning, network telemetry, threat intelligence, alerts and authentication. The API gateway routes each request to the corresponding service. The AI analysis service performs inference with the machine learning models, requests explanations from Gemini and applies the consensus engine, the threat intelligence service queries VirusTotal and the other feeds, and the alerts and webhook service stores incidents and dispatches them to Discord and Slack.

5.1.3.3 Models, External Services and Storage

The PhishGuard model and the locally trained Random Forest model provide inference. The Gemini API provides explanations and VirusTotal provides lookups against more than 72 antivirus engines. Firebase Firestore stores sessions, scan history, alerts and audit logs, supported by an in-memory cache.

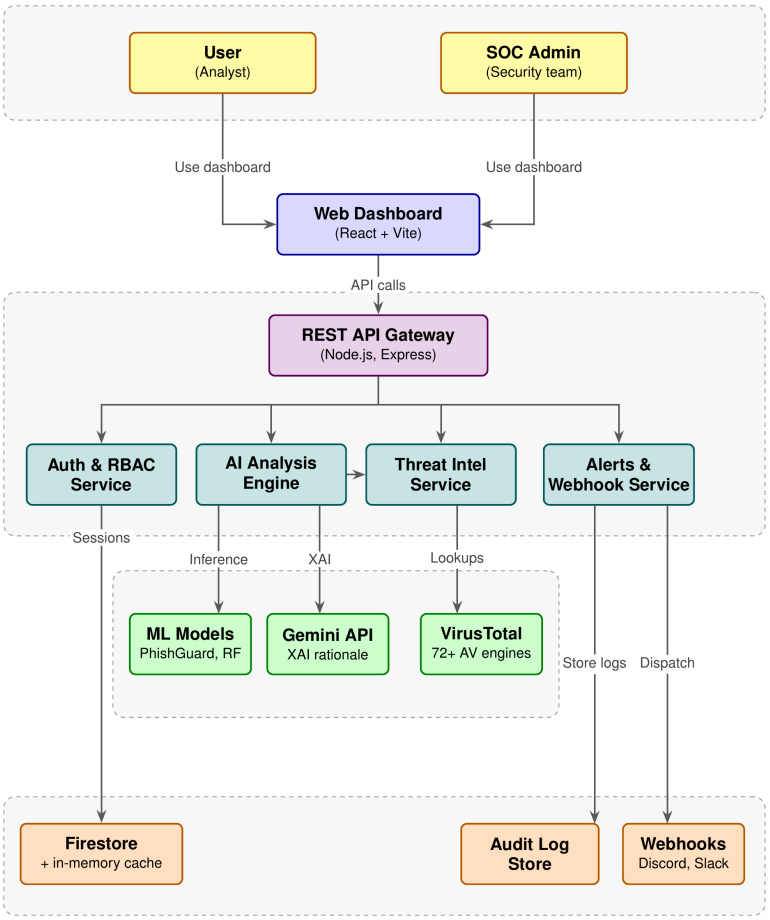


Figure 5.4: System architecture of CyberShield AI

5.1.4 Methodology

The methodology follows the stages of the CyberShield AI pipeline from data preparation and feature extraction through detection, explanation and alerting.

5.1.4.1 Data Preparation

The Random Forest classifier is trained on a balanced corpus of legitimate and malicious URLs prepared with the project's training script `train_model.py`. The PhishGuard model was trained by the project team on a corpus of 235,000 verified URLs and is used through its API.

5.1.4.2 URL Feature Engineering

Fifteen predictors are extracted from every URL. They describe its structure, the distribution of characters, its domain hierarchy, social-engineering keywords and technical markers, as listed in Table 5.1.

Group	Features	Purpose
Structure	URL length, hostname length, path length, query length	Long, complicated URLs often hide the real destination
Token counts	Number of dots, hyphens, slashes, @ signs and digits	Unusual symbols and digits are common in deceptive URLs
Domain hierarchy	Number of subdomains (using `tldextract`)	Deep subdomain chains imitate trusted names
Entropy	Shannon entropy of the URL string	Detects random-looking and generated domains
Keywords	Presence of login, verify, banking, secure, wallet	Social-engineering terms
Technical markers	IP address as host, HTTPS validity, known URL shorteners	Indicators of evasion

Table 5.1: The 15 URL features

The Shannon entropy of a string is computed as

$$H(X) = -\sum_{i=1}^{n} P(x_i) \log_2 P(x_i),$$

where $P(x_i)$ is the relative frequency of the i-th distinct character. High values indicate random-looking strings such as those produced by domain generation algorithms [4, 9].

5.1.4.3 Random Forest Classifier

A Random Forest [7] with 100 decision trees and the Gini impurity criterion is trained with scikit-learn [8] on the 15 features. The trained model is serialised with Joblib and loaded by the back end for inference. Each tree votes on whether the URL is legitimate or malicious and the proportion of votes gives the score of Engine 2.

5.1.4.4 Dual-Engine Consensus

The score of the PhishGuard model (Engine 1) and the score of the Random Forest model (Engine 2) are combined by the consensus engine as

$$\text{Score}_{\text{combined}} = 0.5 \times \text{Score}_{\text{PhishGuard}} + 0.5 \times \text{Score}_{\text{CyberShield}}.$$

If both engines reach the same verdict the result is marked `FULL_AGREEMENT` and is treated as high confidence; if they disagree it is marked `DISCORDANT`, which raises the priority of the case for human review. The combined score is shown as a risk score from 0 to 100.

5.1.4.5 Malware Forensics

When a file is selected, its SHA-256 hash is computed in the browser with the Web Crypto API (`crypto.subtle.digest`), so the file is not uploaded. The hash is sent to VirusTotal, which reports how many of more than 72 antivirus engines detect it. The byte entropy of the file is also calculated, and a value above 7.8 bits per byte indicates packed or encrypted executables.

5.1.4.6 Explainable AI

The indicators produced by the engines, such as lexical features or file properties, are sent to the Google Gemini API, which returns a short summary in plain language describing why the item was flagged and what the analyst should do next.

5.1.4.7 Threat Intelligence and Network Telemetry

The threat intelligence module collects vulnerabilities, indicators and phishing data from public sources, including VirusTotal, PhishTank, the CISA Known Exploited Vulnerabilities catalogue, the MITRE ATT&CK framework and AlienVault OTX, and makes them searchable on the client. The network telemetry module displays simulated traffic with bandwidth, protocol distribution and anomaly counts, refreshed automatically every 10 seconds.

5.1.4.8 Alerting

Detected threats are placed in a live incident queue with a severity of low, medium, high or critical. The administrator sets the Discord and Slack webhook addresses and the minimum severity for notification, and alerts at or above that level are dispatched automatically. Each action is recorded in the audit log.

5.1.4.9 Authentication and Access Control

Users may work as guests or sign in with Google or with an email address and password, verified by a one-time password. Guests can run scans and view results but do not receive

alerts. Signed-in users receive alerts, and the administrator role controls configuration, role assignment and data management.

5.2 Implementation

5.2.1 Web Application Development

The application is organised as seven pages reachable from the side menu, as shown in Figure 6.1 and Figure 6.2.

Page	**Function**
SOC Dashboard	System status, total scans, phishing threats, inference latency, processing rate, telemetry volume, alert severity and recent alerts
Phishing Detection	Two-way phishing detection engine: URL input, the verdict of each engine, consensus result, risk score and scan history log
Malware Forensics	File selection, hash computation, detection results across antivirus engines and summary of the analysis
Network Telemetry	Bandwidth chart, protocol distribution, traffic feed and speed test
Threat Intelligence	Feeds of vulnerabilities and indicators, source list and live synchronisation
Security Alerts	Live incident queue and integration settings for Discord and Slack webhooks and the alert threshold
Settings and Identity	Sign-in, role status, theme selection and data management permissions

Table 5.2: Pages of the web application

The back end validates each request, calls the models and external APIs, applies the consensus logic and returns a JSON response to the front end. The Gemini, VirusTotal and webhook calls are made on the server so that secret keys are not exposed to the browser. Frequently used results are held in an in-memory cache and results, alerts and audit entries are stored in

Firestore.

5.2.2 Testing and Evaluation Plan

The phishing models are evaluated with accuracy and false-positive rate on held-out URLs, and the Random Forest is additionally evaluated by cross-validation. The consensus engine is evaluated by the number of discordant cases and the false positives remaining after human review. Response time is measured for local scoring, hashing, the full dual-engine analysis with explanation, and webhook delivery. The software is tested at unit, integration, end-to-end, performance, security and usability levels.

CHAPTER 6

RESULTS AND DISCUSSION

This chapter describes the evaluation set-up for CyberShield AI and reports the results obtained so far. Values marked belong to experiments or details that have not yet been recorded in this report.

6.1 Evaluation Metrics

For the phishing classifiers, accuracy is the fraction of URLs classified correctly:

$$\text{Accuracy} = \frac{\text{correct predictions}}{\text{all predictions}},$$

and the false-positive rate is the fraction of legitimate URLs wrongly flagged as phishing,

$$\text{FPR} = \frac{FP}{FP + TN}.$$

For the Random Forest, accuracy is estimated by cross-validation, in which the data is divided into folds and each fold in turn is used for testing while the others are used for training. For the system as a whole, response time and delivery time are measured in milliseconds or seconds.

6.2 Experimental Set-up

Item	**Value**
Engine 1	PhishGuard model, trained on 235,000 verified URLs, used through its API
Engine 2	Random Forest, 100 trees, Gini criterion, 15 features
Consensus	Equal-weight average of the two scores
Malware hashing	SHA-256 in the browser (Web Crypto API)
Hash lookup	VirusTotal, 72 or more antivirus engines
Explanations	Google Gemini API
Alerts	Discord and Slack webhooks
Telemetry refresh	Every 10 seconds
Random Forest corpus (source, size, split)	
Test hardware and network	

Table 6.1: Experimental set-up

6.3 Detection Performance

Model / configuration	**Result**
PhishGuard model (Engine 1)	97.4% accuracy on the corpus of 235,000 verified URLs
Random Forest (Engine 2)	95.8% cross-validated accuracy with the 15 features
Dual-engine consensus	False-positive rate below 1.8%, with discordant cases flagged for human triage

Table 6.2: Detection performance of the phishing engines

Each engine alone achieves high accuracy. Combining them lowers the false-positive rate further, because a legitimate URL wrongly flagged by one engine is usually not flagged by the other, and the disagreement itself is reported to the analyst instead of being hidden.

6.4 System Latency and Alert Delivery

Operation	**Measured time**
Local lexical heuristic scoring	below 12 ms
Complete dual-engine analysis with Gemini explanation	1.4 to 2.2 s
In-browser SHA-256 hashing (files up to 50 MB)	below 35 ms
Webhook delivery to Discord / Slack	below 400 ms
Background refresh interval	10 s

Table 6.3: System latency and alert delivery

All measured values meet the performance targets set in NFR-01: hashing under 50 ms and dual-engine classification with explanation under 2.5 seconds.

6.5 Screenshots of the Application

Figures 6.1 and 6.2 show the working application. The first figure shows the sign-in dialogue, the SOC dashboard, the two-way phishing detection page and the malware hash analysis page. The second shows the network telemetry page, the threat intelligence feeds, the security alerts page with webhook settings and the settings and identity page.

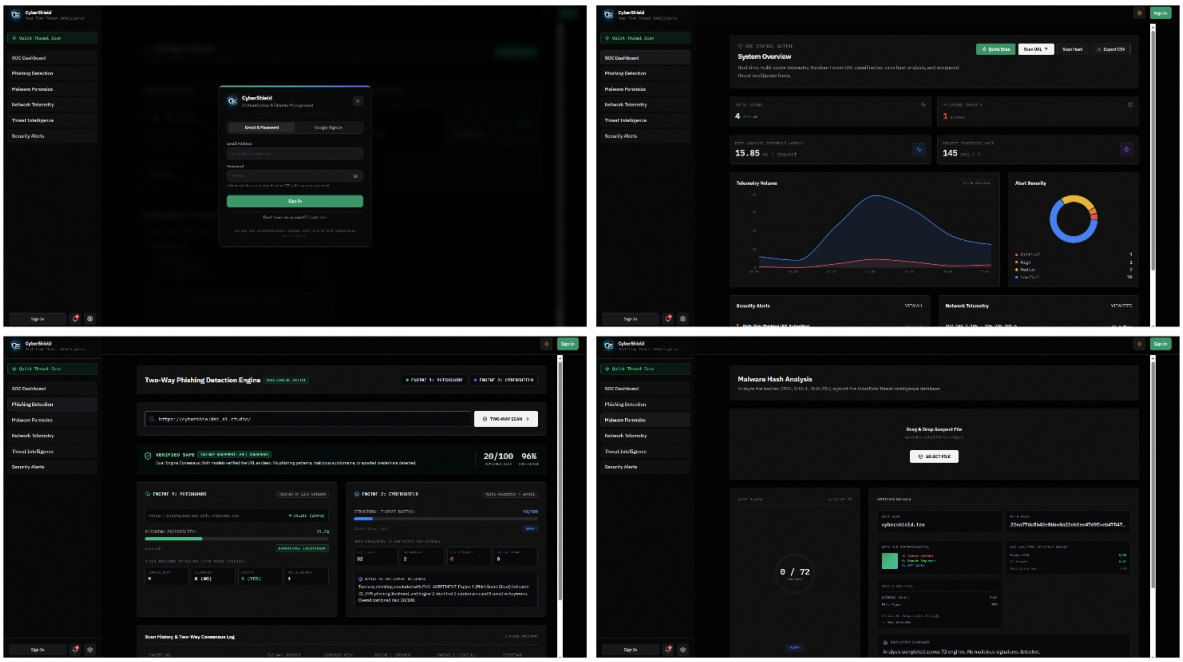


Figure 6.1: Application screens: sign-in, SOC dashboard, phishing detection and malware forensics

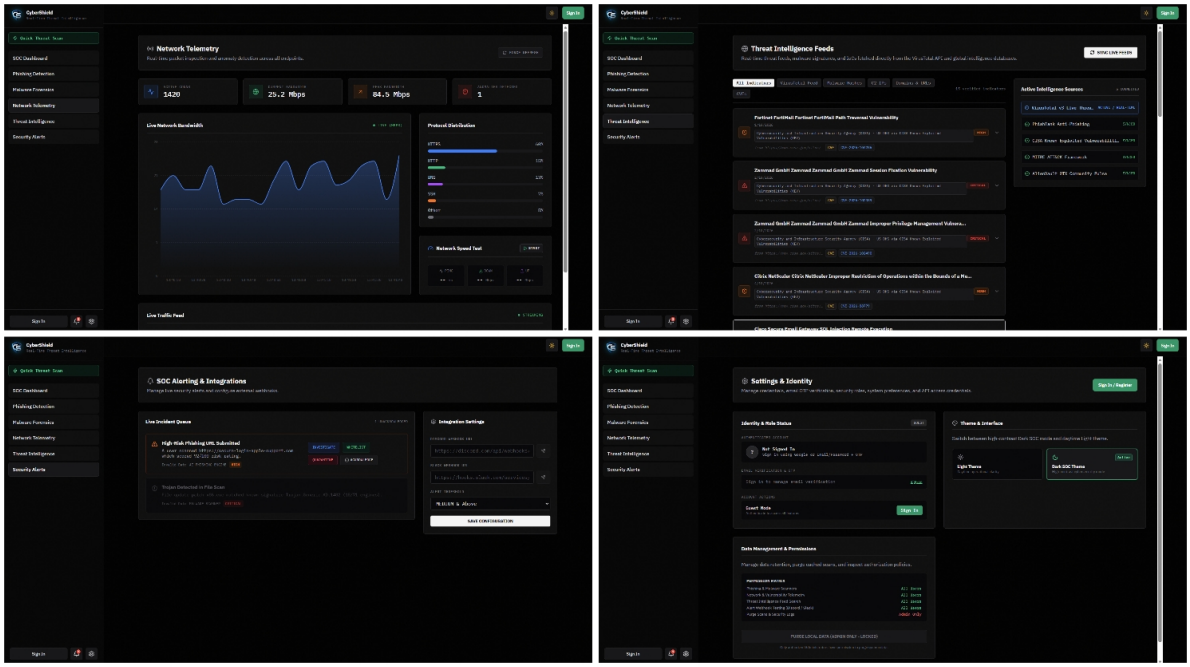


Figure 6.2: Application screens: network telemetry, threat intelligence, security alerts and settings

6.6 Testing and Validation of the Application

Sl No	Procedure	Expected result	Status
1	Scan a legitimate URL as a guest	Both engines agree; low risk score shown	Done
2	Scan a known phishing URL	Both engines flag it; high risk and explanation shown	Done
3	Scan a URL on which the engines disagree	Result marked discordant for review	Done
4	Select a file for malware analysis	SHA-256 shown; file is not uploaded	Done
5	Sign in with Google and with email and password	Session created; alerts enabled	Done
6	Use the platform as a guest	Scans work; no alerts are delivered	Done
7	Configure a webhook and trigger a high-severity alert	Message reaches Discord / Slack	Done
8	Attempt an admin action as an analyst	Action refused by role-based access control	Done
9	Disable an external API	Result of the remaining service shown without crash	Done

Table 6.4: Functional and system test cases

6.7 Discussion

The results indicate that a lightweight platform can combine detection, explanation and alerting while remaining responsive. The two phishing engines give high accuracy on their own and the consensus step reduces false positives, which matters in a SOC where alert fatigue is a real problem. Computing hashes in the browser keeps the cost of malware checks low and protects the confidentiality of files. The Gemini explanation turns an opaque score into guidance that

a junior analyst can act on, and the webhook alerts shorten the time between detection and notification.

Several limitations remain. The network telemetry is simulated, so anomaly detection on real traffic has not yet been evaluated. The hash lookup can only identify files already known to the antivirus engines, so new malware may pass unnoticed until it is analysed elsewhere. The Gemini explanations depend on an external service and may vary between requests. The accuracy figures come from the project's own datasets and should be confirmed on URLs from different sources before the platform is relied on in practice. Guest users receive no alerts, so continuous monitoring requires signing in.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This report presented CyberShield AI, a web-based platform that brings together phishing detection, malware forensics, network telemetry, threat intelligence and automated alerting. Phishing URLs are checked by two independent engines, the PhishGuard model and a Random Forest classifier on 15 features, whose verdicts are cross-verified by a consensus engine. Files are hashed in the browser and matched against more than 72 antivirus engines, the Gemini API explains flagged threats in plain language, and Discord and Slack webhooks notify the team. The platform can be used as a guest or after signing in, with role-based access control for administrators. The literature survey showed that individual detectors are accurate but are rarely packaged with explanations, alerting and a unified interface, which is the gap that CyberShield AI addresses.

In Phase I, the problem was defined, the literature reviewed, and the requirements, architecture, methodology and a working implementation established. The results so far show high detection accuracy, a reduced false-positive rate and response times suited to interactive use. The next stage is to complete the remaining experiments, test the platform on data from other sources and carry out user testing with analysts.

7.2 Future Scope

- **Automated sandbox execution:** browser sandboxing to detonate suspicious URLs and scripts and observe their behaviour.
- **Live feed ingestion:** continuous collection from PhishTank, OpenPhish and URLhaus.
- **Custom rules:** support for uploading YARA and Sigma rules for local file classification.
- **Mobile and PWA application:** progressive web app support with push notifications to incident responders.
- **Real network telemetry:** replacing simulated traffic with data from real sensors or logs

and evaluating anomaly detection on it.

- **Adaptive models:** periodic retraining of the phishing models with analyst-verified verdicts to reduce concept drift.
- **Alerts for guests:** an optional lightweight notification channel for guest users.

REFERENCES

[1] M. Misiek and T. Hyla, "XGBoost-Based URL Phishing Detection Method With Cross-Dataset Validation," *IEEE Access*, vol. 14, 2026.

[2] T. Koide, H. Nakano, and D. Chiba, "ChatSpamDetector: Large Language Model-Based Phishing Email Detection," *IEEE Access*, vol. 14, 2026.

[3] L. Aldhuwayhi and A. Alromih, "A Dynamic Phishing URL Detection System Integrating Advanced Machine Learning and Crowdsourced Threat Intelligence," *IEEE Access*, 2026.

[4] J. D. Duarte et al., "Machine Learning for Early Detection of Phishing URLs in Parked Domains," *IEEE Access*, vol. 13, 2025.

[5] A. Elgarduh et al., "Hybrid Feature Image Mapping (HFIM): A Multi-Channel Image Approach for Malware Detection," *IEEE Access*, 2026.

[6] R. Gutierrez et al., "Application of Deep Learning Models for Real-Time Automatic Malware Detection," *IEEE Access*, vol. 12, 2024.

[7] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[9] C. E. Shannon, "A Mathematical Theory of Communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.